Lesson: Pointers and Arrays

CSE 4107 : Structured Programming I

Shahriar Ivan

Introduction

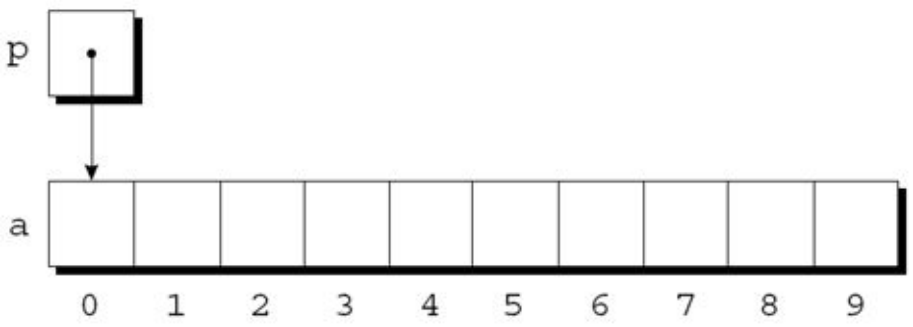
- C allows us to perform *arithmetic*—***addition*** and ***subtraction***—on pointers to array elements
- This leads to an alternative way of processing arrays in which *pointers* take the place of array subscripts
- The relationship between pointers and arrays in C is a close one
- Understanding this relationship is critical for mastering C

Pointer Arithmetic

- Pointers can point to array elements:

```
int a[10], *p;
p = &a[0];
```

- A graphical representation:

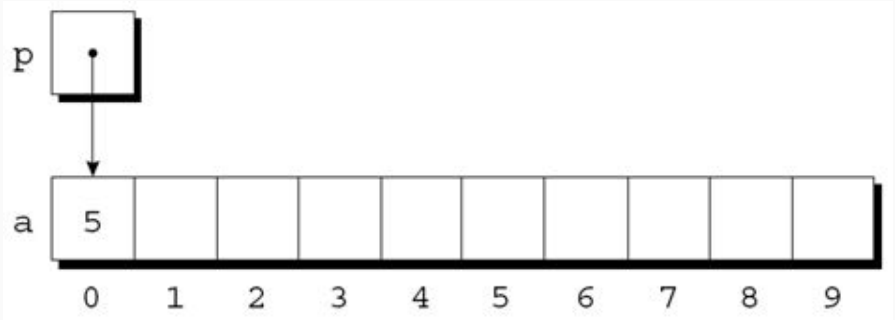


Pointer Arithmetic

- We can now access a[0] through p; for example, we can store the value 5 in a[0] by writing

```
*p = 5;
```

- An updated picture:



Pointer Arithmetic

- If p points to an element of an array a, the other elements of a can be accessed by performing ***pointer arithmetic*** (or ***address arithmetic***) on p
- C supports three (and only three) forms of *pointer arithmetic*:
  - Adding an integer to a pointer
  - Subtracting an integer from a pointer
  - Subtracting one pointer from another

Pointer Arithmetic

Adding an Integer to a Pointer

- Adding an integer *j* to a pointer *p* yields a pointer to the element *j* places after the one that *p* points to
- More precisely, if *p* points to the array element `a[i]`, then `p + j` points to `a[i+j]`
- Assume that the following declarations are in effect:

```
int a[10], *p, *q, i;
```

Pointer Arithmetic

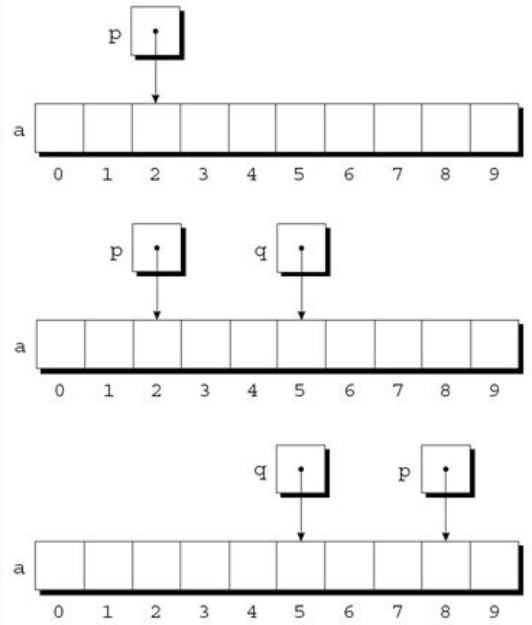
Adding an Integer to a Pointer

- Example of pointer addition:

```
p = &a[2];

q = p + 3;

p += 6;
```



Pointer Arithmetic

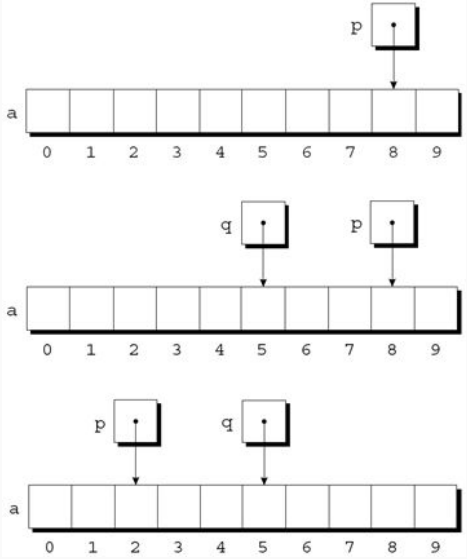
Subtracting an Integer from a Pointer

- If *p* points to a[i], then p - j points to a[i-j]
- Example:

```
p = &a[8];

q = p - 3;

p -= 6;
```

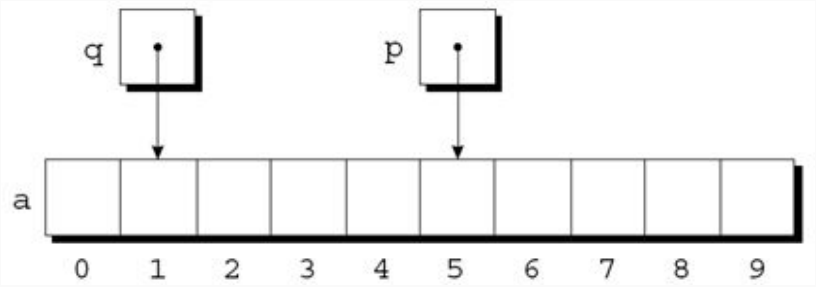


Pointer Arithmetic

Subtracting an Integer from a Pointer

- When one pointer is subtracted from another, the result is the ***distance*** (measured in array elements) between the pointers
- If *p* points to a[i] and *q* points to a[j], then p - q is equal to i - j
- Example:

```
p = &a[5];
q = &a[1];
i = p - q;   /* i is 4 */
i = q - p;   /* i is -4 */
```



Pointer Arithmetic

Subtracting an Integer from a Pointer

- Operations that cause undefined behavior:
  - Performing arithmetic on a pointer that doesn't point to an array element
  - Subtracting pointers unless both point to elements of the same array

```
int a[10], b[10], *p, *q, i, j;
p = &a[0];
i = p - q;                    /*** WRONG ***/
q = &b[0];
j = p - q;                    /*** WRONG ***/
```

Pointer Arithmetic

Comparing Pointers

- Pointers can be compared using the relational operators (`<`, `<=`, `>`, `>=`) and the equality operators (`==` and `!=`)
  - Using relational operators is meaningful only for pointers to elements of the same array
- The outcome of the comparison depends on the relative positions of the two elements in the array
- After the assignments

```
p = &a[5];
q = &a[1];
```

the value of p <= q is 0 and the value of p >= q is 1

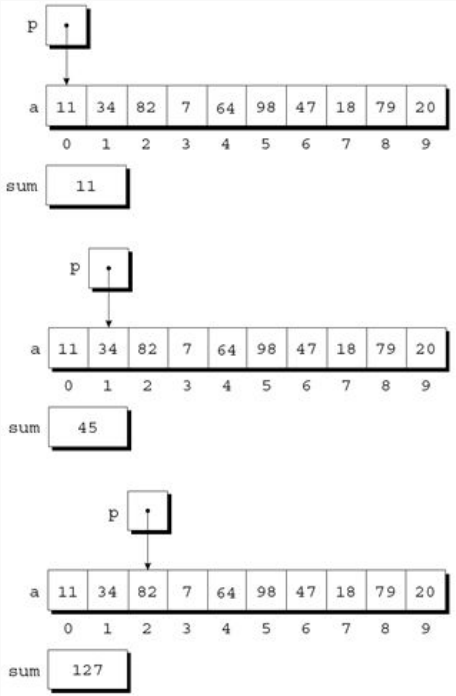
Using Pointers for Array Processing

- Pointer arithmetic allows us to visit the elements of an array by repeatedly incrementing a pointer variable
- A loop that sums the elements of an array *a*:

```
#define N 10
…
int a[N], sum, *p;
…
sum = 0;
for (p = &a[0]; p < &a[N]; p++)
        sum += *p;
```

Using Pointers for Array Processing

- At the end of the first iteration:
- At the end of the second iteration:
- At the end of the third iteration:



Using Pointers for Array Processing

- The condition `p < &a[N]` in the for statement deserves special mention
- It's legal to apply the address operator to `a[N]`, even though this element doesn't exist
- Pointer arithmetic may save execution time
- However, some C compilers produce better code for loops that rely on *subscripting*

Using Pointers for Array Processing

*Comparing the * and ++ Operators*

- Modify an array element and advance to the next

```
*p++ = j;              // same as a[i++] = j;
```

Expression	***Meaning***
*p++ or *(p++)	Value of expression: *p. Increment p later
(*p)++	Value of expression: *p. Increment *p later
*++p or *(++p)	Increment p first. Value of expression: *p after increment
++*p or ++(*p)	Increment *p first. Value of expression: *p after increment

Using Array Name as a Pointer

- Pointer arithmetic is one way in which arrays and pointers are related
- Another key relationship:

  The name of an array can be used as a pointer to the first element in the array

- This relationship simplifies pointer arithmetic and makes both arrays and pointers more versatile

Using Array Name as a Pointer

- Suppose that a is declared as follows:

```
int a[10];
```

- Examples of using a as a pointer:

```
*a = 7;                 /* stores 7 in a[0] */
*(a+1) = 12;            /* stores 12 in a[1] */
```

- In general, a + i is the same as &a[i]
  - Both represent a pointer to element i of a
- Also, *(a+i) is equivalent to a[i]
  - Both represent element i itself

Using Array Name as a Pointer

- The fact that an array name can serve as a pointer makes it easier to write loops that step through an array
- Original loop:

```
for (p = &a[0]; p < &a[N]; p++)
    sum += *p;
```

- Simplified version:

```
for (p = a; p < a + N; p++)
    sum += *p;
```

Using Array Name as a Pointer

- Cannot be assigned a new value

```
a = p;              /*** WRONG ***/
```

- Cannot make it point elsewhere

```
while (*a != 0)
  a++;              /*** WRONG ***/
```

- Solution: Copy a into a pointer variable

```
p = a;
while (*p != 0)
  p++;
```

Programming Task

Reversing a Series of Numbers (Revisited)

- The reverse.c program of Chapter 8 reads 10 numbers, then writes the numbers in ***reverse*** order
- The original program stores the numbers in an array, with *subscripting* used to access elements of the array
- reverse3.c is a new version of the program in which *subscripting* has been replaced with pointer arithmetic

Using Array Name as a Pointer

- C allows us to subscript a pointer as though it were an array name:

```
#define N 10
…
int a[N], i, sum = 0, *p = a;
        …
for (i = 0; i < N; i++)
      sum += p[i];
```

**The compiler treats p[i] as *(p+i)

Array Arguments (Revisited)

- When passed to a function, an array name is treated as a pointer

```
int find_largest(int a[], int n){
        int i, max;
        max = a[0];
        for (i = 1; i < n; i++)
                if (a[i] > max)
                        max = a[i];
        return max;
}
```

- A call of *find_largest*:

  largest = find_largest(b, N);

  This call causes a pointer to the first element of b to be assigned to a;

Array Arguments (Revisited)

- As opposed to ordinary variables, when array is passed
  - Pointer is passed instead of copying
  - Editing array variable can change array in argument

```
void store_zeros(int a[], int n){
        int i;
        for (i = 0; i < n; i++)
                a[i] = 0;
}
```

- To prevent change, use const

```
int find_largest(const int a[], int n);
```

Array Arguments (Revisited)

- The time required to pass an array to a function doesn't depend on the size of the array ⇒ No copy is made
- An array parameter can be declared as pointer ⇒ Compiler treats them as identical

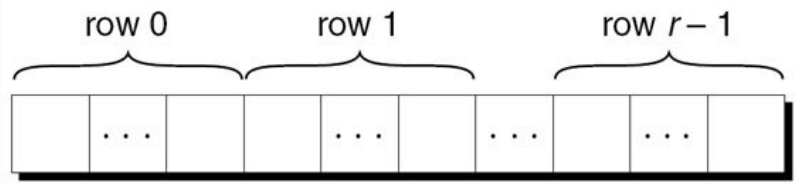
```
int find_largest(int *a, int n);
```

- Not the same for variable declaration
  - Sets aside space for 10 integers `int a[10];`
  - Allocates space for a pointer variable `int *a;`

Pointers and Multidimensional Arrays

Processing the Elements of a Multidimensional Array

- Just as pointers can point to elements of one-dimensional arrays, they can also point to elements of multidimensional arrays
- Layout of an array with *r* rows:



- If *p* initially points to the element in *row 0*, *column 0*, we can visit every element in the array by incrementing *p* repeatedly

Pointers and Multidimensional Arrays

Processing the Elements of a Multidimensional Array

- Consider the problem of initializing all elements of the following array to zero:

```
int a[NUM_ROWS][NUM_COLS];
```

- The obvious technique would be to use nested for loops:

```
int row, col;
…
for (row = 0; row < NUM_ROWS; row++)
    for (col = 0; col < NUM_COLS; col++)
        a[row][col] = 0;
```

Pointers and Multidimensional Arrays

Processing the Elements of a Multidimensional Array

- If we view a as a one-dimensional array of integers, a single loop is sufficient:

```
int *p;
…
for (p = &a[0][0]; p <= &a[NUM_ROWS-1][NUM_COLS-1]; p++)
  *p = 0;
```

- Works with most C compilers

Pointers and Multidimensional Arrays

Processing the Rows of a Multidimensional Array

- A pointer variable *p* can also be used for processing the elements in just one *row* of a two-dimensional array
- To visit the elements of *row i,* we'd initialize *p* to point to element *0* in *row i* in the array a:

```
p = &a[i][0];
```

or we could simply write,

```
p = a[i];
```

Pointers and Multidimensional Arrays

Processing the Rows of a Multidimensional Array

int A[2][3];

A[0], A[1] will be 1D-arrays of 3 integers

So, we can not write, int *p = A;

int (*)p[3] = A;

Here, A ⟶ &A[0]

*A ⟶ A[0] ⟶ &A[0][0]

A+1 ⟶ &A[1]

*(A+1) ⟶ A[1] ⟶ &A[1][0]

*(A+1)+2 ⟶ A[1]+2 ⟶ &A[1][2]

A[0] ⟶	1	2	3
A[1] ⟶	4	5	6